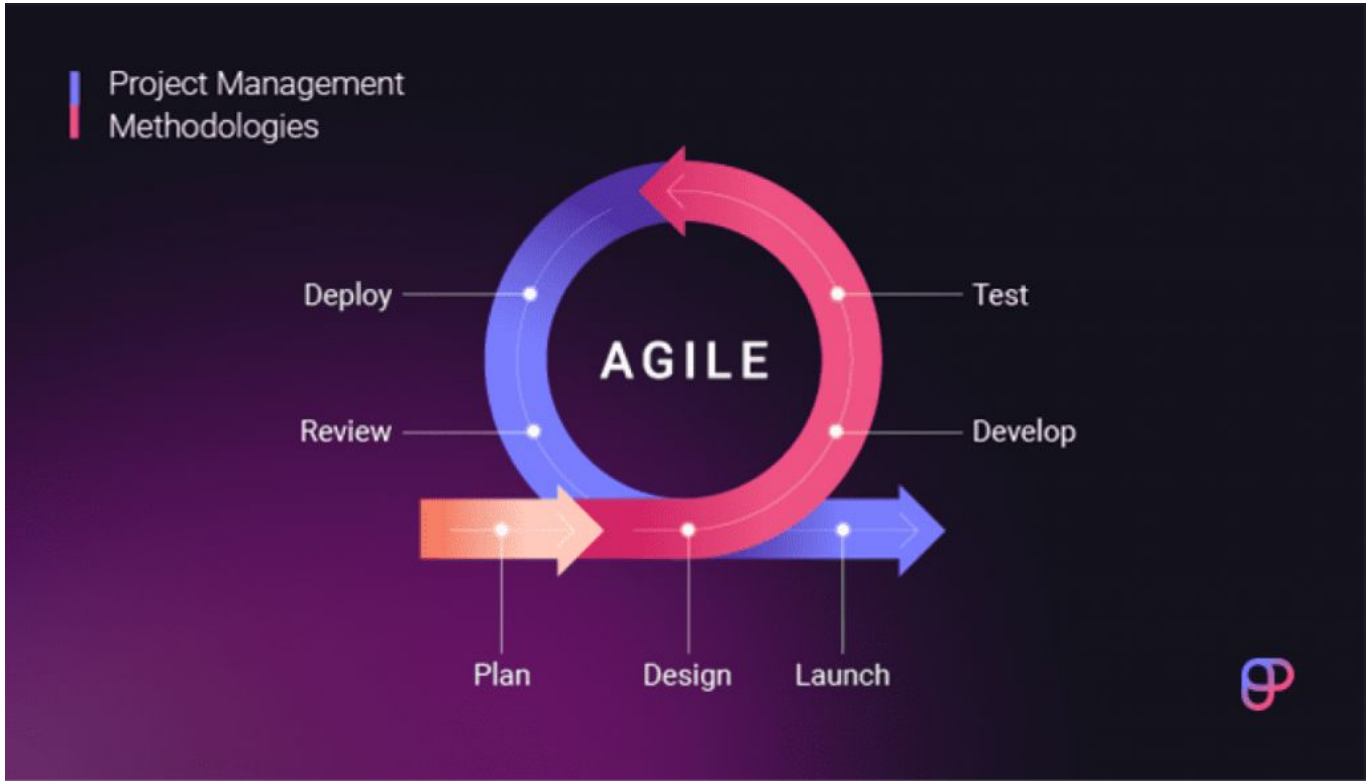
Unit-2
Scrum and Sprint

SEM: VI

Course: AGILE METHODOLOGY FOR PROJECT DEVELOPMENT

Course Code: 23OE0027

Faculty: Prof. LALIT ASHUTOSH



Jira Software

Textbook & Authors

TEXT BOOKS:

1. Methodology for the Design of Agile Product Development Networks, Riesener, Michael; Rebentisch, Eric; Doelle, Christian; Kuhn, Maximilian; Brockmann, Soeren, 2019, Elsevier.
2. Roman Pichler – *Agile Product Management with Scrum: Creating Products that Customers Love*, Addison-Wesley, 2010.

REFERENCE BOOKS:

1. Agile Product Development: How to Design Innovative Products That Create Customer Value, Tathagat Varma, Springer Nature,1st ed. Edition, Apress.
2. Donald G. Reinertsen – The Principles of Product Development Flow: Second Generation Lean Product Development, Celeritas Publishing, 2009. (Bridges Agile & Lean principles.)

Overview of Syllabus

- UNIT – I
  - Introduction to Agile
- UNIT – II
  - Scrum and Sprint
- UNIT – III
  - Agile Project Management
  - UNIT – IV
  - Agile Testing
  - UNIT – V
  - Agile Software Development Case Studies

DISCLAIMER

The content (text, image, and graphics) used in this slide are adopted from many sources for academic purposes. Broadly, the sources have been given due credit appropriately. However, there is a chance of missing out some original primary sources. The authors of this material do not claim any copyright of such material.

Scrum

Scrum is an Agile framework used to manage and deliver complex products through short, iterative cycles called **Sprints** (usually 1–4 weeks).

It focuses on:

- Transparency
- Inspection
- Adaptation
- Continuous delivery of value

Scrum is defined in the **Scrum Guide** by **Ken Schwaber** and **Jeff Sutherland**.

Scrum Process

The **Scrum process** is an Agile framework used to develop, deliver, and sustain complex products through iterative and incremental work. It is built around transparency, inspection, and adaptation.

1. Product Backlog Creation

- The **Product Owner** creates and maintains the **Product Backlog**.
- It contains prioritized features, enhancements, bug fixes, and technical work.
- Items are usually written as **user stories**.

2. Sprint Planning

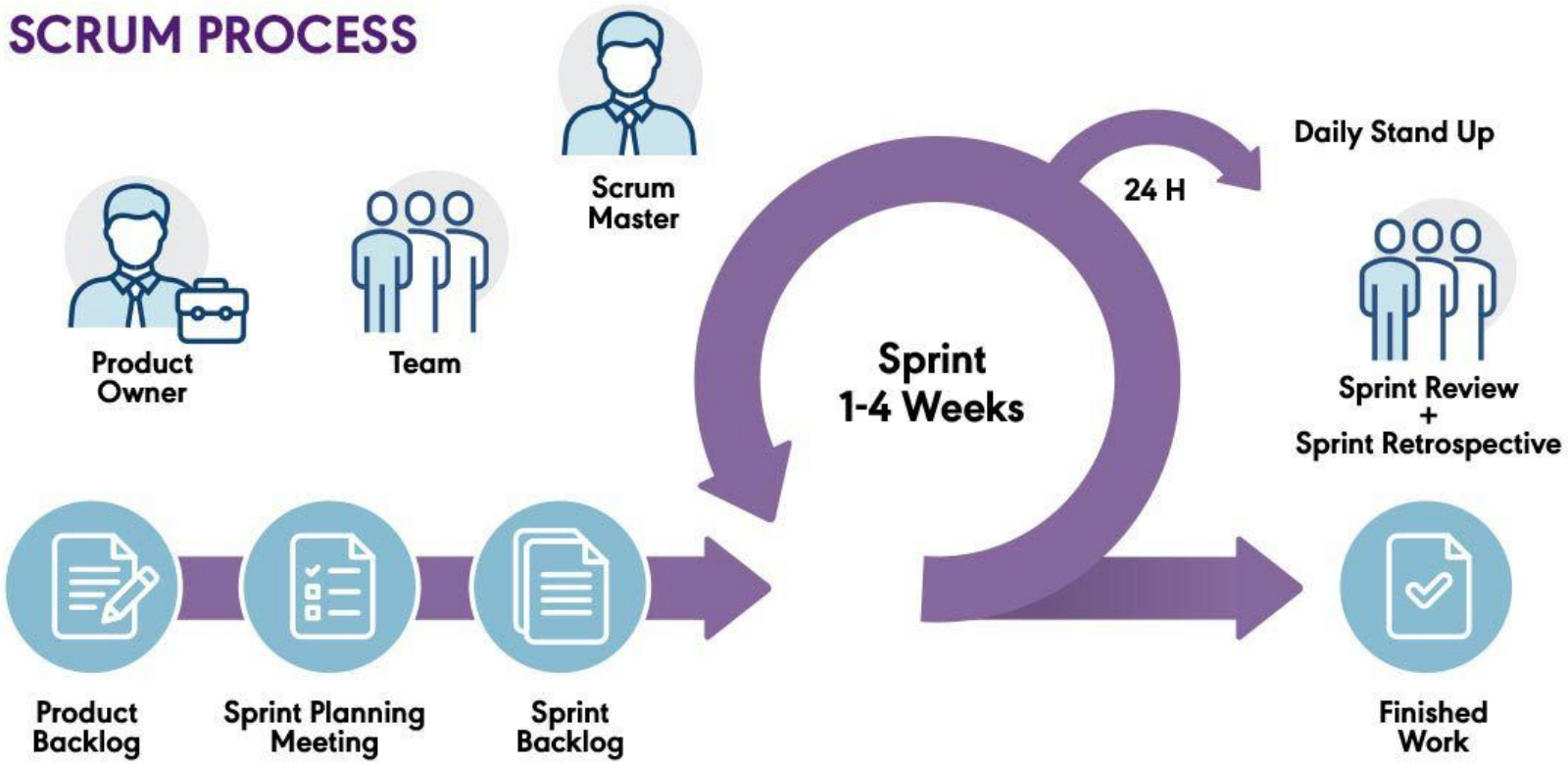
- Conducted at the beginning of each Sprint.
- The Scrum Team selects items from the Product Backlog.
- The team defines:
  - **Sprint Goal**
  - **Sprint Backlog**
  - Plan for delivering selected work.
- Sprint duration: typically **1–4 weeks**.

3. Sprint Execution

- The Development Team works on selected backlog items.
- Work is self-organized.
- Focus is on delivering a **potentially shippable product increment**.

4. Daily Scrum (Daily Stand-up)

- 15-minute daily meeting.
- Team members discuss:
  - What was done yesterday?
  - What will be done today?
  - Any impediments?
- Helps track progress toward Sprint Goal.


SCRUM PROCESS
Scrum Master
Product Owner
Team
Daily Stand Up
24 H
Sprint 1-4 Weeks
Sprint Review + Sprint Retrospective
Product Backlog
Sprint Planning Meeting
Sprint Backlog
Finished Work

Scrum Roles

1. Product Owner (PO)

- Owns and manages the **Product Backlog**
- Defines product vision and priorities
- Maximizes product value
- Accepts or rejects completed work
- Represents stakeholders

2. Scrum Master

- Servant leader and facilitator
- Ensures Scrum practices are followed
- Removes impediments
- Coaches the team and organization in Agile principles
- Facilitates Scrum events

3. Development Team (Scrum Team)

- Cross-functional and self-organizing
- Delivers a potentially shippable product increment
- Responsible for quality and meeting Sprint goals

Other Roles (Not Official Scrum Roles but Often Present)

4. Release Manager

- Plans and coordinates product releases
- Manages release schedules and deployment
- Works closely with Product Owner and stakeholders

5. Project Manager

- In traditional setups: manages scope, time, cost
- In Scrum environments: responsibilities are often distributed between PO and Scrum Master

6. Product Manager

- Focuses on product strategy and market needs
- Works closely with Product Owner
- May define long-term roadmap

7. Architect

- Guides technical direction
- Ensures system design consistency
- Supports team in making architectural decisions

Difference Between Product Owner and Scrum Master

Aspect	Product Owner (PO)	Scrum Master (SM)
Primary Focus	Maximizing product value	Ensuring Scrum is followed effectively
Responsibility	*What* to build	*How* the team works
Ownership	Product Backlog	Scrum process
Works Closely With	Stakeholders, customers, business	Development team, organization
Decision Authority	Prioritizes backlog, accepts/rejects work	No authority over product decisions; facilitates process
Success Measured By	Product value, ROI, customer satisfaction	Team effectiveness, process improvement
Role Type	Business-focused	Process-focused (servant leader)

Scrum Events (Ceremonies)

1. Sprint

- Time-boxed iteration (1–4 weeks)
- Produces a usable product increment

2. Sprint Planning

- Defines Sprint Goal
- Selects backlog items
- Creates Sprint Backlog

3. Daily Scrum

- 15-minute daily meeting
- Tracks progress and identifies impediments

4. Sprint Review

- Demonstration of completed work
- Stakeholder feedback session

5. Sprint Retrospective

- Team reflection meeting
- Focus on process improvement

Scrum Artifacts

Scrum Artifacts are key information **tools used in Scrum that provide transparency and help track work and progress.**

There are **three main Scrum Artifacts**:

1. Product Backlog

An ordered list of everything needed in the product.

Owned by: Product Owner

Contains:

- User stories
- Features
- Enhancements
- Bug fixes

Purpose:

- Shows *what needs to be built*
- Prioritized based on business value

Example:

- Login feature
- Payment integration
- Fix search bug

2. Sprint Backlog

List of items selected from Product Backlog for the current Sprint, plus plan to deliver them.

Owned by: Development Team

Contains:

- Selected user stories
- Tasks
- Sprint Goal

Purpose:

- Shows *what team will do in this Sprint*

3. Increment

The usable, working version of the product that is completed during a Sprint.

Owned by: Development Team

Characteristics:

- Usable
- Tested
- Meets Definition of Done

Purpose:

- Shows *actual progress*

Example:

- Working login feature ready to use

imagine your team is building a **mobile banking app**.

Sprint 1 Increment:

- Login screen completed
- Login tested
- Ready to use

This working login feature = **Increment**

Product Inception

Product Inception is the initial phase of an Agile project where the team defines the product vision, goals, scope, stakeholders, and initial backlog before starting development.

It answers the question: "**What are we going to build and why?**"

Objectives of Product Inception

- Define product vision
- Identify stakeholders
- Understand business goals
- Create initial product backlog
- Define scope and boundaries
- Estimate effort and timeline

Activities in Product Inception

1. Product Vision Creation

- Defines purpose of the product
- Explains problem being solved
- Gives overall direction

Example:

Create a mobile app for online food ordering.

2. Identify Stakeholders

Stakeholders include:

- Customers
- Users
- Product Owner
- Developers
- Managers

3. Define Goals and Objectives

- Business goals
- User goals
- Success criteria

4. Create Initial Product Backlog

- List of features
- Written as user stories

Example:

- User login
- Search product
- Payment system

5. Initial Estimation

- Rough effort estimation
- Helps in planning releases

6. Define Scope

- What is included
- What is excluded

Importance of Product Inception

- Provides clear direction
- Reduces confusion
- Aligns team and stakeholders
- Improves planning

Stakeholders (in Scrum / Agile)

Stakeholders are individuals, groups, or organizations who have an interest in the product or are affected by it. They can influence the product or are impacted by its success or failure.

Types of Stakeholders

Internal Stakeholders

- Product Owner
- Scrum Master
- Development Team
- Project Manager
- Product Manager
- Company Management

External Stakeholders

- Customers
- End Users
- Clients
- Sponsors
- Investors
- Suppliers

Role of Stakeholders

- Provide requirements
- Give feedback on product
- Help define product vision
- Suggest improvements
- Review product during Sprint Review

Importance of Stakeholders

- Ensure product meets user needs
- Help in decision making
- Provide support and funding
- Improve product quality

Example

In an **Online Shopping System:**

- Customers → Buy products
- Admin → Manage products
- Company → Earn profit
- Developers → Build system

All are stakeholders.

Initial Backlog Creation

Initial backlog creation usually refers to building the **first version of a Product Backlog** in Agile / Scrum.

This is the prioritized list of features, enhancements, fixes, and tasks needed to deliver a product.

It is done during the **Product Inception phase**.

Who creates it?

- Mainly **Product Owner**
- With help from:
  - Stakeholders
  - Scrum Team
  - Customers

What does Initial Backlog contain?

- User stories
- Features
- Functional requirements
- Non-functional requirements
- Bug fixes (if known)

Steps in Initial Backlog Creation

1. Define the Product Vision

Start with understanding:

- What problem are you solving?
- Who are the users?
- What is the goal of the product?

Output: Product vision statement

Example:

"Build a mobile app that helps farmers track crop health."

2. Identify High-Level Features (Epics)

List major functionalities.

Examples:

- User login
- Dashboard
- Reports
- Notifications

These are called **Epics**

3. Break Epics into User Stories

Convert features into **user stories**

Format:

As a [user], I want [feature], so that [benefit]

Example:

- As a user, I want to login so that I can access my account
- As a user, I want to view dashboard so that I can see summary

4. Add Acceptance Criteria

Define when story will complete.

Example:

User Story: Login

Acceptance Criteria:

- User enters email
- User enters password
- User clicks login
- Dashboard opens

5. Estimate Stories

Use story points:

Common scale:

- 1 – Very small
- 2 – Small
- 3 – Medium
- 5 – Large
- 8 – Very large

6. Prioritize the Backlog

Order based on:

- Business value
- Urgency
- Risk
- Dependencies

Highest value = top

7. Create Backlog in Tool

Common tools:

- Jira
- Azure DevOps
- Trello
- Excel

8. Backlog is NEVER Final

It keeps evolving.

You will:

- Add new stories
- Modify stories
- Remove stories
- Re-prioritize

This is called **Backlog Refinement**

Agile Requirements

Agile Requirements are the techniques used in Agile development to identify, document, organize, and manage what the system should do, in a simple, flexible, and user-focused way.

Unlike traditional methods, Agile requirements are:

- Lightweight (no heavy documents)
- Written from the user's perspective
- Continuously refined
- Managed in a backlog
- Delivered in small increments (sprints)

Instead of big requirement documents, Agile uses:

- User Personas
- User Stories
- Backlogs
- Acceptance Criteria
- Story Mapping

User Personas

A **User Persona** is a **fictional representation of a real user**, based on research and assumptions, used to understand the user's needs, goals, and behavior.

A User Persona represents the **type of person who will use the system**.

User Persona = Representation of target user.

Why User Personas are used in Agile

User Personas help the team to:

- Understand the users
- Focus on user needs
- Write better user stories
- Design user-friendly systems

Without personas, teams build systems based on assumptions.

Key elements of a User Persona

A persona usually includes:

- Name
- Age
- Job / Role
- Goals
- Needs
- Problems
- Skills / behavior

Example of User Persona

Example: Online Shopping System

Name: Rahul Sharma
Age: 28
Profession: Software Engineer

Goals:

- Buy products quickly
- Easy payment

Problems:

- Slow websites
- Complicated checkout

Needs:

- Fast checkout
- Easy navigation

How Personas are used in Agile

Personas help write **User Stories**

Example:

Persona: Rahul

User Story:

As Rahul
I want to add items to cart
So that I can purchase products

Benefits of User Personas

- Improves requirement clarity
- Helps understand users
- Improves product design
- Helps write better user stories
- Improves team understanding

User Stories

A **User Story** is a **short, simple description of a requirement written from the user's perspective**, describing what the user wants and why.

A User Story describes **a feature from the user's point of view**.

User Story = Requirement written from user perspective

Standard Format (Very important)

User stories follow this format:

As a [type of user]

I want [feature]

So that [benefit]

Example : Online Shopping System

As a customer

I want to add items to my cart

So that I can purchase them

Example : College Portal

As a student

I want to register for courses

So that I can attend classes

Components of a User Story

User Story has 3 parts:

1. **User** → Who
2. **Feature** → What
3. **Benefit** → Why

Why User Stories are used

User stories help to:

- Understand user requirements
- Focus on user needs
- Replace long requirement documents
- Make requirements simple and clear
- Support Agile development

User Story Example with Acceptance Criteria

User Story:

As a student
I want to log in
So that I can access my account

Acceptance Criteria:

- Student enters username and password
- System verifies credentials
- Login successful

NOTE:

User Stories are stored in: **Product Backlog**

Story Mapping

Story Mapping is a technique used to organize user stories in a visual way to understand the system, user workflow, and overall product functionality.

Story Mapping is arranging user stories in order of how the user uses the system.

It shows:

- Big picture of the product
- User activities
- User tasks
- Development priorities

Why Story Mapping is used

Story Mapping helps to:

- Understand the complete system
- See how users interact with the system
- Organize user stories
- Plan releases and sprints
- Identify important features

Basic Structure of Story Mapping

Story Map has 3 levels:

1. User Activities (High level – Big tasks)

Example:

- Login
- Search product
- Add to cart
- Make payment

2. User Tasks (Sub-tasks)

Example under Login:

- Enter username
- Enter password
- Click login

3. User Stories (Detailed requirements)

Example:

As a user
I want to enter username
So that I can login

Example: Online Shopping System Story Map

Step 1: Activities

- Browse products
- Select product
- Add to cart
- Checkout
- Payment

Step 2: Tasks under Add to cart

- Click Add button
- View cart
- Update quantity

Step 3: User Story

As a customer
I want to add items to cart
So that I can purchase

3Cs of User Stories

The **3Cs of User Stories** represent the **three important components required to fully understand a user story.**

The 3Cs are:

- **Card**
- **Conversation**
- **Confirmation**

The 3Cs help ensure that a user story is properly written, discussed, and tested.

1. Card

The **Card** is the **written user story**.

It contains a short description of the requirement.

Usually written on:

- Index card
- Sticky note
- Agile tool (Jira, etc.)

Example:

As a student
I want to log in
So that I can access my account

This written statement is the **Card**

2. Conversation

The **Conversation** is the **discussion between team members and stakeholders** about the user story.

This discussion clarifies:

- What exactly is needed
- How it will work
- Any doubts

Participants:

- Developers
- Testers
- Product Owner
- Customer

Purpose:

To clearly understand the requirement

3. Confirmation

The **Confirmation** is the **acceptance criteria**.

It confirms that:

The user story is correctly implemented.

Example (Confirmation / Acceptance Criteria):

- User enters username
- User enters password
- System logs in successfully

Complete Example (All 3Cs together)

Card:

As a user
I want to log in
So that I can access my account

Conversation:

Discussion:

- Should login use email?
- What if password is wrong?

Confirmation:

Acceptance Criteria:

- Valid username and password
- Successful login
- Error message for wrong password

Why 3Cs are important

3Cs help to:

- Improve requirement clarity
- Avoid misunderstandings
- Ensure proper implementation
- Improve communication

INVEST Principle

The **INVEST Principle** is a **set of guidelines used to check the quality of User Stories** in Agile.

It helps ensure that user stories are **clear, complete, and useful.**

INVEST stands for:

- **I** – Independent
- **N** – Negotiable
- **V** – Valuable
- **E** – Estimable
- **S** – Small
- **T** – Testable

1. I – Independent

The user story should be **independent**, not dependent on other stories.

This means:

It can be developed separately.

Example:

Good:

As a user
I want to login
So that I can access my account

Bad:

Login depends on Profile creation story

Example (Independent)

User Story:

As a user
I want to login
So that I can access my account

This is independent because:

- Login can be developed alone
- It does not depend on another story

Example (Dependent)

Imagine two stories:

Story 1:

As a user
I want to create profile

Story 2:

As a user
I want to login

If login only works after profile creation is finished, then **Login story depends on Profile story**

This makes it **dependent**, which is not ideal.

2. N – Negotiable

The user story should be **flexible**, and details can be discussed.

It should not be fixed like a contract.

Team and customer can discuss and improve it.

3. V – Valuable

The user story should provide **value to the user**.

It must be useful.

Example:

Good:

As a customer
I want to make payment
So that I can purchase product

Provides value.

Example (Valuable)

User Story:

As a customer
I want to make payment
So that I can purchase product

Why valuable?

Because:

- Customer can buy product
- It helps customer complete their goal

4. E – Estimable

The user story should be **easy to estimate**.

Team should be able to estimate:

- Time
- Effort

If story is too big, break it.

Example (Estimable)

User Story:

As a user
I want to login using email and password
So that I can access my account

Why estimable?

- Clear requirements
- Team can estimate time easily (example: 2 days)

Example (Not Estimable)

User Story:

As a user
I want full banking system

Why not estimable?

- Too big
- Too vague
- Cannot estimate time

5. S – Small

The user story should be **small and simple**.

It should be completed in one sprint.

Bad:

Build entire shopping system

Good:

Add item to cart

6. T – Testable

The user story should be **testable**.

It must have acceptance criteria.

Tester should verify it.

Example:

Login success or failure can be tested.

Why INVEST is important

INVEST helps to:

- Improve user story quality
- Avoid confusion
- Make stories clear
- Help in testing
- Help in development planning

Acceptance Criteria

Acceptance Criteria are the conditions that a user story must satisfy to be considered complete and accepted.

Acceptance Criteria tell when a User Story is done and correct.

They act as **confirmation** in the **3Cs**.

Why Acceptance Criteria are important

They help to:

- Define clear requirements
- Avoid confusion
- Help developers understand what to build
- Help testers know what to test
- Confirm completion of user story

Relationship with User Story

User Story = What user wants
Acceptance Criteria = When it will complete

Example 1: Login System

User Story:

As a student
I want to login
So that I can access my account

Acceptance Criteria:

- Student enters valid username and password
- System allows login
- Error message shown for invalid login

Example 2: ATM Withdrawal

User Story:

As a user
I want to withdraw money
So that I can use cash

Acceptance Criteria:

- User enters correct PIN
- User selects amount
- Cash is dispensed
- Receipt is printed

Sprint

A **Sprint** is a **fixed time period in Agile during which a set of user stories is developed, tested, and delivered.**

Sprint is a short development cycle where the team completes selected work.

Duration of Sprint

Usually:

- **1 week to 4 weeks**
- Most common: **2 weeks**

Sprint duration is fixed.

Sprint Cycle (Steps)

1. Sprint Planning

Team selects work from Product Backlog

2. Development

Develop features

3. Testing

Test features

4. Sprint Review

Show completed work to customer

5. Sprint Retrospective

Discuss improvements

Key characteristics of Sprint

- Fixed time period
- Delivers working software
- Contains user stories
- Repeats until project completion

Real-life example (Easy classroom example)

Building a Mobile App

Sprint 1:

- Login

Sprint 2:

- Messaging

Sprint 3:

- Notifications

Each Sprint delivers features.

Product Backlog

Product Backlog is a list of all requirements, features, and user stories that need to be developed in a project.

Product Backlog is the main list of all work to be done.

It contains everything the product needs.

Key points about Product Backlog

- It contains **User Stories**
- It is **managed by Product Owner**
- It is **continuously updated**
- It is **prioritized** (important items at top)
- It is the **source for Sprint Planning**

Contents of Product Backlog

Product Backlog may include:

- User Stories
- Features
- Bug fixes
- Improvements

Characteristics of Product Backlog

Product Backlog is:

- Dynamic (changes anytime)
- Prioritized
- Ordered list

Example

Mobile App Product Backlog:

- Login
- Send message
- Upload photo
- Video call

Why Product Backlog is important

It helps to:

- Organize requirements
- Plan work
- Track progress

Backlog Grooming

Backlog Grooming (also called **Backlog Refinement**) is the process of **reviewing, updating, and improving the Product Backlog.**

Backlog Grooming is cleaning and organizing the Product Backlog.

Why Backlog Grooming is needed

It helps to:

- Keep backlog updated
- Add new requirements
- Remove unnecessary items
- Clarify user stories
- Set priorities
- Prepare for next sprint

What activities happen in Backlog Grooming

During backlog grooming, the team:

- Reviews user stories
- Adds new stories
- Removes unwanted stories
- Breaks large stories into smaller stories
- Adds acceptance criteria
- Estimates effort
- Changes priority

Example

Product Backlog (Before Grooming):

1. Login
2. Shopping system (too big)
3. Payment

After Backlog Grooming:

1. Login
2. Add to cart
3. Checkout
4. Payment

Large story broken into smaller stories.

Who participates in Backlog Grooming

- Product Owner
- Scrum Master
- Developers
- Testers

It is a team activity.

Test First Development (TFD)

Test First Development (TFD) is a software development technique in which **test cases are written before writing the actual program code**.

The main goal is to ensure the code works correctly by testing it from the beginning.

Steps in Test First Development

1. **Write a test case**
2. **Run the test**
3. **Write the code**
4. **Run the test again**
5. **Refactor the code**

Steps in Test First Development

1. Write a Test Case

- Create a test for a small piece of functionality.
- Define **expected behavior** clearly.
- At this point, the actual function may not exist.

Example:

Test: Function `add(2,3)` should return `5`

2. Run the Test → It Fails

- Since the code is not written, the test fails.
- This is called the **Red phase**

Purpose:

- Confirms the test is working correctly
- Confirms the functionality doesn't exist yet

3. Write Code to Pass the Test

- Write the **minimum code required** to make the test pass.
- Do not add extra features yet.

Example:

```
def add(a, b):
  return a + b
```

This is called the **Green phase**

4. Run the Test Again → It Passes

- Now the test succeeds.
- Confirms the functionality works as expected.

5. Refactor the Code

- Improve code quality
- Remove duplication
- Improve readability
- Optimize if needed

Without breaking the test

This is called the **Refactor phase**

This cycle is called:

🔴 **RED** → 🟢 **GREEN** → 🔵 **REFACTOR**

Example: Function to Check Even Number

Step 1: Write the Test Case (FIRST)

We want a function:

`isEven(number)`

Expected behavior:

- `isEven(4)` → True
- `isEven(5)` → False

Test Case (Python)

```
def test_is_even():
    assert isEven(4) == True
```

Step 2: Run the Test → It Fails

Error:

```
NameError: name 'isEven' is not defined
```

Reason:

Function is not written yet.

This is the **RED phase**

Step 3: Write Code to Pass the Test

Now write minimum code:

```
def isEven(number):
    return number % 2 == 0
```

Step 4: Run Test Again → It Passes

Output:

```
Test Passed
```

This is the **GREEN phase**

Step 5: Refactor (Optional)

Code is already simple, so no change needed.

This is **REFACTOR phase**

Why TDD is Used

Benefits:

- Better design
- Fewer bugs
- Cleaner code
- Safer refactoring
- Higher confidence

Pair Programming

Pair Programming is a software development technique where **two programmers work together on one computer** to write code.

Two Roles in Pair Programming

1. Driver

- Writes the code
- Controls keyboard and mouse
- Focuses on implementation

2. Navigator

- Reviews the code
- Finds mistakes
- Suggests improvements
- Thinks about overall logic

They **switch roles frequently**

Example

Task: Write function to add two numbers

Driver:

Writes code

```
def add(a, b):
    return a + b
```

Navigator:

Checks and suggests:

- Add test case
- Handle negative numbers

Advantages

- Fewer bugs
- Better code quality
- Knowledge sharing
- Faster learning
- Better team collaboration

Disadvantages

- Needs two developers
- May increase cost
- Requires good teamwork

Code Reviews

Code Review is a systematic checking of code written by a developer by another developer to detect errors and improve code quality.

Example

Developer A writes code:

```
def divide(a, b):
    return a / b
```

Reviewer checks and suggests:

- Handle divide by zero

Improved code:

```
def divide(a, b):
    if b == 0:
        return "Cannot divide by zero"
    return a / b
```

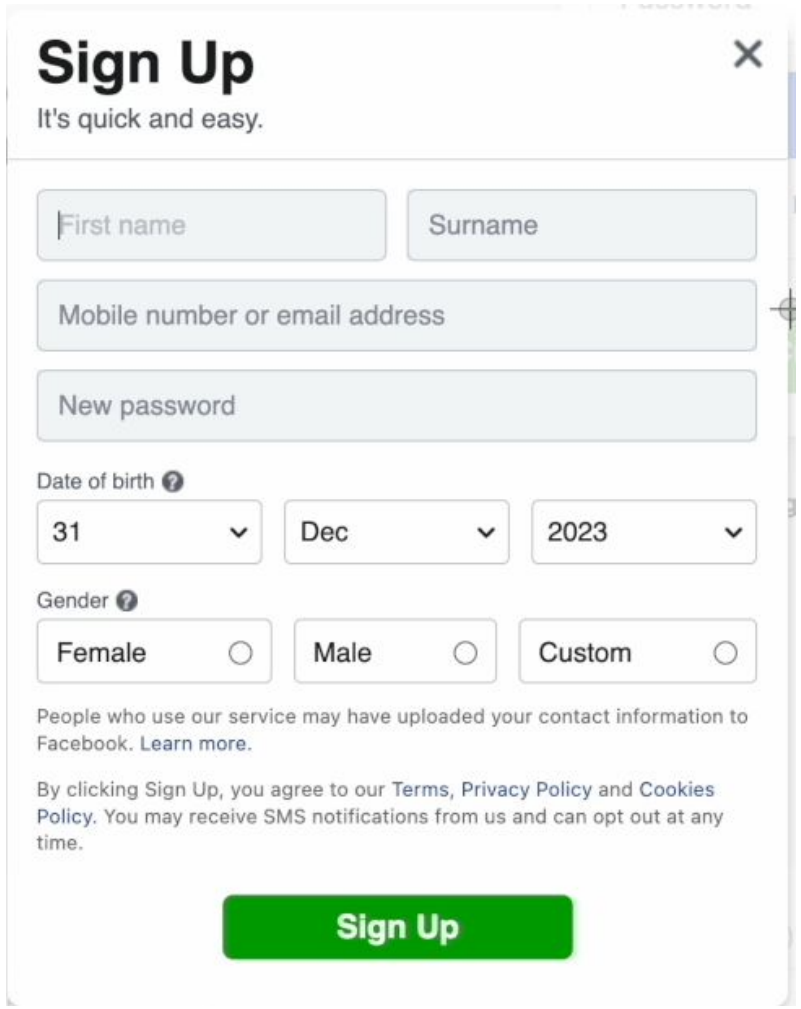
Advantages

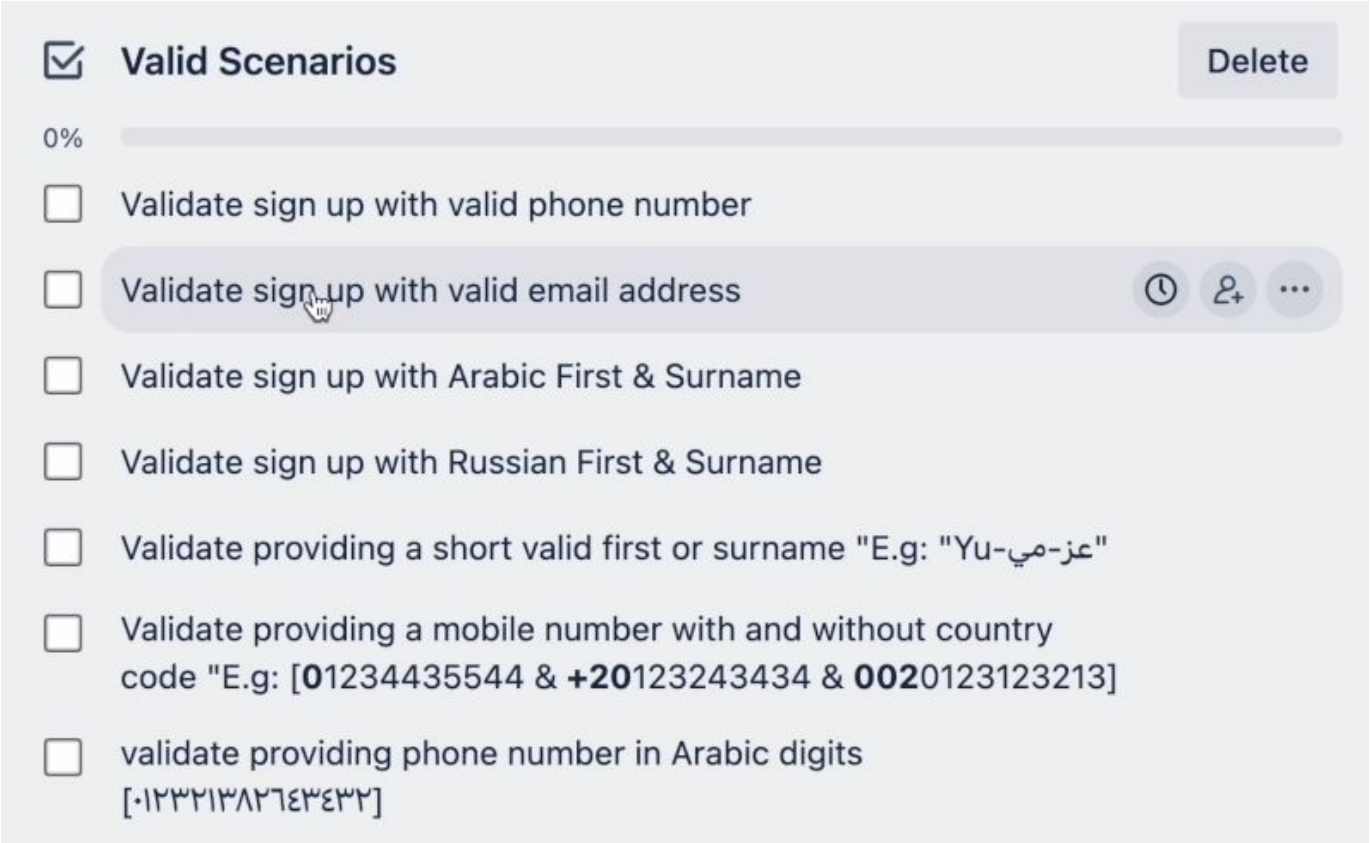
- Finds bugs early
- Improves code quality
- Improves team knowledge
- Ensures coding standards

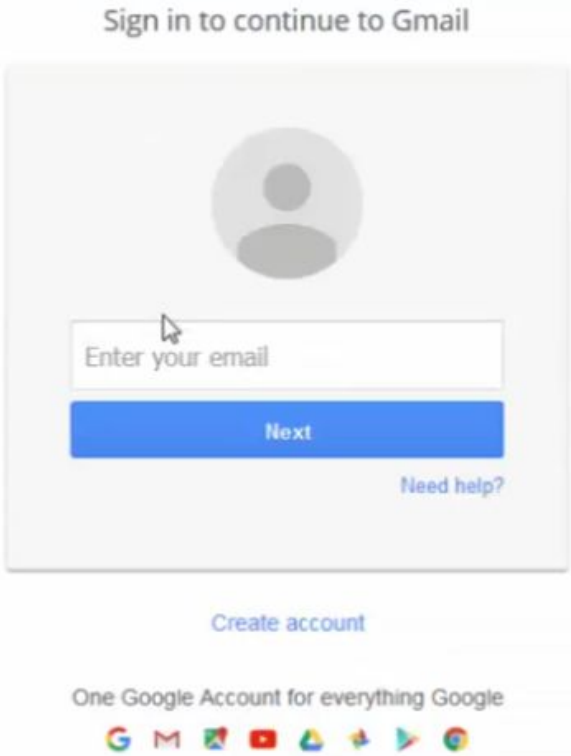
Disadvantages

- Takes time
- Needs skilled reviewers

Write Test Scenario







Testing Gmail Login Functionality with Valid userid and password

TestCaseID	Test Title	Description	Test Case Steps	Expected Result	Actual Result	Status
MTC01	Gmail Login Test	Testing login functionality of Gmail.	1. Open www.gmail.com on webbrowser.	Gmail home page is opened	Gmail home page opened.	Pass
			2. Type Valid gmail id in mail id section.			
			3. Click on next	One webpage which gives an option to enter password.	Got a web page providing option for password	Pass
			4. Enter correct password			
			5. Click on Sign in.	User is logged in to Gmail	Logged in to Gmail.	Pass
						Pass

THANK YOU